壁仞通用命令行工具测试平台 V1.0

1. 简介 3
2. 功能模块介绍 3
2.1. 检查和更新 firmware 3
2.1.1 检查、更新 firmware 模块设计 3
2.1.1.1 更新 firmware 前端模块设计 3
2.1.1.2 远程处理模块 4
2.1.1.3 检查 firmware 版本模块设计 6
2.1.1.4 更新 firmware 模块设计 6
2.1.2 检查、更新 firmware 模块流程图 7
2.2. 从远程安装 full stack 包 8
2.2.1 安装 full-stack 模块设计 8
2.2.1.1 更新 full-stack 前端设计 8
2.2.1.2 远程处理模块 8
2.2.1.3 更新 full-stack 模块设计 8
2.2.2 安装 full-stack 模块流程图 9
2.3. 上传测试文件 10
2.3.1 上传测试文件模块设计 10
2.3.1.1 上传测试文件前端设计 10
2.3.1.2 上传文件内容设计 10
2.3.1.3 上传测试文件 run case 模块设计 11
2.3.2 上传测试文件模块流程图 12
2.4. 执行测试并同时运行监控程序 13
2.4.1 执行测试程序模块设计 13
2.4.1.1 excel case 处理模块设计 13
2.4.1.2 生成 report 并保存 14
2.4.2 在执行 case 的同时运行监控程序 14
2.4.2.1 monitor 模块设计 14
2.4.2.2 case 运行过程中的调用 15
2.4.3 测试执行并监控模块流程图 15
2.5. 展示测试结果 16
2.5.1 测试结果展示模块设计 16
2.5.1.1 展示 case 匹配结果 16
2.5.1.1.1 展示结果表格 16
2.5.1.1.2 展示结果饼图 16
2.5.1.2 展示 monitor 结果 17
2.5.2 测试执行并监控模块流程图 17
2.6. 返回测试过程中是否有系统 error 18
2.6.1 Error 展示模块设计 18
2.6.1.1 展示是否有系统 error 的模块设计 18
2.6.2 测试执行并监控模块流程图 18
3. 简单用例 20

1. 简介

壁仞通用命令行测试平台是基于 Flask 框架编写的用于测试命令行工具的系统平台。由于命令行工具的共同点在于：在命令行输入命令，获得期待的返回。由此设计了该测试系统。只要将输入的命令和期待的返回写入到 excel 表格中，就能在该平台上进行任意 remote 的自动化验证。操作步骤简单，且结果细致直观。

该系统主要功能有

- 检查和更新 firmware
- 从远程安装 full-stack 包
- 上传测试文件
- 执行测试并同时运行监控程序
- 展示测试结果及监控结果
- 返回测试过程中是否有系统 error

该系统亮点：

- 通用性体现在所有直入直出的命令行工具都可以使用该系统进程测试。
- 在指定的 ip 上一键进行更新 firmware 和安装 full-stack 的操作
- 能在任意 remote ip 上执行自动化 case
- 使用 pandas 批量处理测试 case，大大提高了测试速度和效率
- 使用 excel 管理命令行用例，使测试 case 粒度更细

2. 功能模块介绍

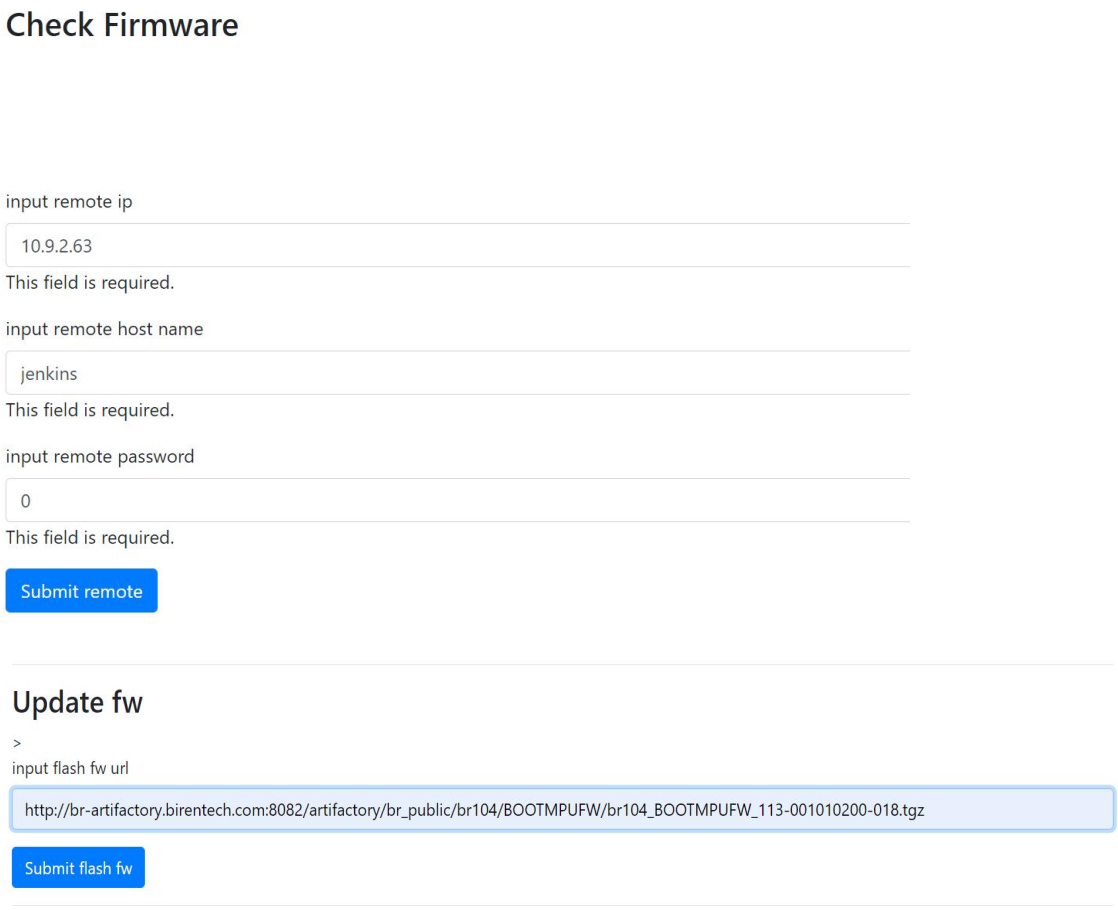
2.1. 检查和更新 firmware

当在 web 端 dashboard 页面输入要查询的 remote ip 后，将调用该机器上的 brsmi 工具查询 firmware 版本，若该机器上没有预先安装 brsmi 工具，可以使用 br_debug 查询，该平台后端将会自动安装 br_debug。
若需要更新 firmware，需在 web 页面上输入对应的 firmware url。点击更新后将会在远端执行更新程序，并将结果或报错信息展示在前端。

2.1.1 检查、更新 firmware 模块设计

2.1.1.1 更新 firmware 前端模块设计

从前端接收 remote ip，host name 及 password，submit 之后，check firmware 的结果将展示在该页面。



2.1.1.2 远程处理模块

由于测试不一定是在同一台机器中，需要支持不同的 remote 上也能进行测试的功能，由此设计了远程处理模块，包括以下内容：

- 建立远程链接

```
def do_ssh_connect(self) -> paramiko.SSHClient:
    self.ssh_client = paramiko.SSHClient()
    # allow to connect host witch isn't in known_host file
    self.ssh_client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
    self.ssh_client.connect(hostname=self.hostname, username=self.username, password=self.password)
```

- 在 remote 和 server 之间收发文件

```python
def send_single_file_to_remote(self, src_file:str, dst_file:str) -> None:
    sftp = paramiko.SFTPClient.from_transport(self.ssh_client.get_transport())
    #sftp = ssh.open_sftp() # or this
    sftp.put(src_file,dst_file)
    sftp.close()

def get_single_file_from_remote(self, src_file:str, dst_file:str) -> None:
    t=paramiko.Transport((self.hostname,22))
    t.connect(username=self.username, password=self.password)
    sftp = paramiko.SFTPClient.from_transport(t)
    sftp.get(src_file,dst_file)
    t.close()

def send_dir_to_remote(self, src:str, dst:str):
    sftp = paramiko.SFTPClient.from_transport(self.ssh_client.get_transport())

    try:
        local_files = sftp.listdir(src)
        for file in local_files:
            local_file = src + file
            remote_file = dst + file
            sftp.put(local_file, remote_file)
    except IOError as e:
        return ("remote_path or local_path is not exist, ", e)

    sftp.close()

def get_dir_from_remote(self, src:str, dst:str):
    sftp = paramiko.SFTPClient.from_transport(self.ssh_client.get_transport())

    try:
        remote_files = sftp.listdir(src)
        print(remote_files)
        for file in remote_files:
            print(file)
            remote_file = src + file
            local_file = dst + file
            sftp.get(remote_file, local_file)
    except IOError as e:
        return ("remote_path or local_path is not exist, ", e)

    sftp.close()
```

- 远程执行命令功能模块（执行交互命令）

```python
def interact_with_remote(self,cmd_list,echo_list):
    trans=paramiko.Transport((self.hostname,22))
    trans.connect(username=self.username, password=self.password)
    channel = trans.open_session()
    channel.settimeout(7200)
    channel.get_pty()
    channel.invoke_shell()
    content = []
    for cmd in cmd_list:
        ct = self.channel_send(channel,cmd)
        content.append(ct)
    content_echo = self.echo_cmd(channel,echo_list,"$")
    channel.close()
    trans.close()
    return content, content_echo
```

2.1.1.3 检查 firmware 版本模块设计

将通过远程交互模块，将查询命令发送到 remote ip，并在 remote 上执行命令，返回结果到 server。

```
def check_fw_by_debugger(self) -> list:
    card_res_list = self.gb.multi_card_run("--fw_version")
    return card_res_list

def check_fw_by_brsmi(self):
    self.load_origin_kmd()
    cmd = "brsmi gpu query |grep firmware"
    content = self.cr.exe_wait(cmd)
    return content
```

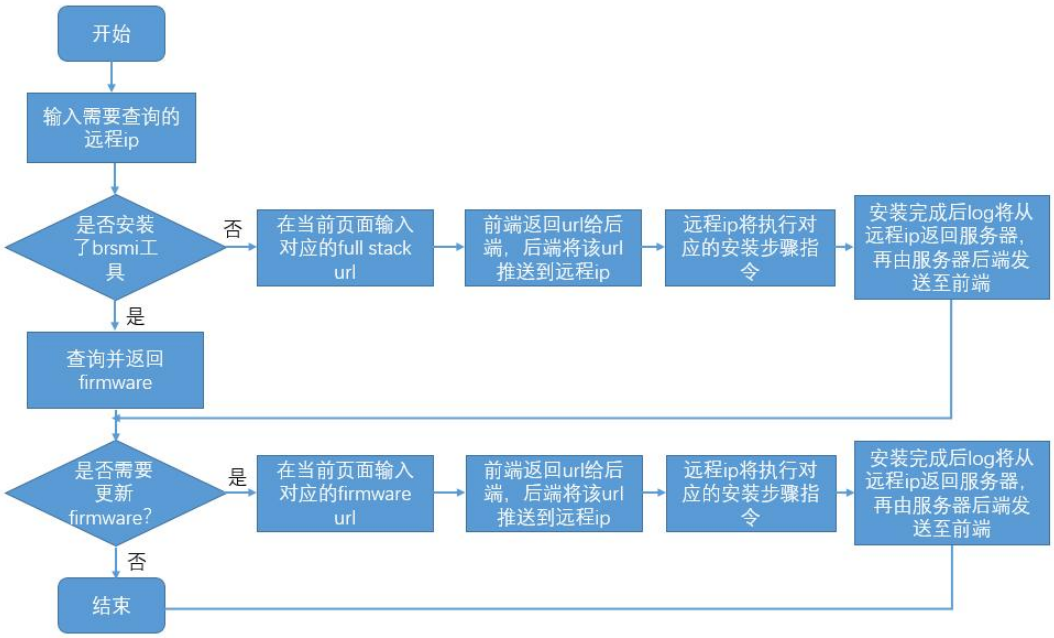
2.1.1.4 更新 firmware 模块设计

将从前端接收 firmware 的 url，在后端处理更新逻辑

```
def update_fw(self,fw_http) -> None:
    get_cmd = f"cd {PKG_SPACE}; wget {fw_http}; wget {self.tools}"
    fw_tar = self.rp.reg_match(r"br104_BOOTMPUFW_113-\d{9}-\d{3}.tgz",fw_http)[0]
    tool_tar = self.rp.reg_match(r"brflash-brfru_.+?.tar.gz",self.tools)[0]
    tar_cmd = f"cd {PKG_SPACE}; tar xvf {fw_tar}; tar xvf {tool_tar}"
    fw_dir = self.rp.reg_match(r"113-\d{9}-\d{3}",fw_tar)[0]
    bin_file_cmd = f"mv {PKG_SPACE}/{fw_dir}/*_signed.bin {PKG_SPACE}/template.bin"
    self.cr.exe_on_terminal(get_cmd)
    self.cr.exe_on_terminal(tar_cmd)
    self.cr.exe_on_terminal(bin_file_cmd)

    if self.cr.exe_on_terminal("lsmod |grep biren") != 1:
        self.cr.exe_on_terminal("sudo rmmod biren")
    for i in range(self.card.card_num):
        self.cr.exe_on_terminal(f"sudo {PKG_SPACE}/brflash-brfru/brflash -r -i {i}")
        self.cr.exe_on_terminal(f"sudo {PKG_SPACE}/brflash-brfru/brflash -c u -f {PKG_SPACE}/template.bin -i {i}")
        self.cr.exe_on_terminal(f"sudo {PKG_SPACE}/brflash-brfru/brflash -r -i {i}")
        time.sleep(3)
```

2.1.2 检查、更新 firmware 模块流程图



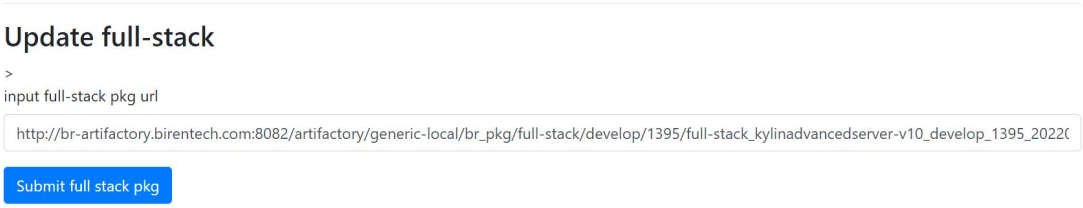
2.2. 从远程安装 full stack 包

若 remote 机器上没有 suvs 组件及其依赖的其他模块，则无法进行测试。若没有 brsmi 及其相关组件，则无法进行监控。所以从 web 端输入远程 ip 和需要更新的 full stack url，将在该远程机器上更新 full stack。使用 full-stack 一键安装 biren 软件栈，更加高效且便利。

2.2.1 安装 full-stack 模块设计

2.2.1.1 更新 full-stack 前端设计

输入要更新的 full-stack url，点击 submit 之后，该 url 将发送到后端进行处理。



2.2.1.2 远程处理模块

远程处理模块可复用检查更新 firmware 中所提到的远程处理。

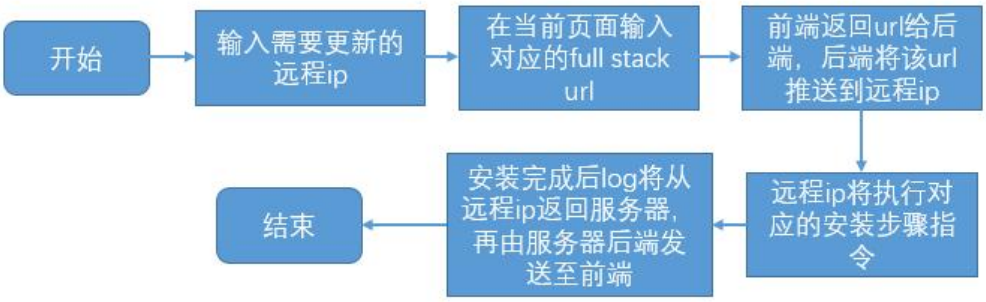
2.2.1.3 更新 full-stack 模块设计

使用正则匹配，解压从 url 获取的压缩包，并进入解压目录安装 full-stack。

```
def load_origin_kmd(self) -> None:
    load_kmd_cmd = "sudo modprobe biren"
    self.cr.exe_on_terminal(load_kmd_cmd)


def load_full_stack(self,pkg_http) -> None:
    get_pkg_cmd = f"cd {PKG_SPACE}; wget {pkg_http}"
    tar_pkg = self.rp.reg_match(r"full-stack_.+",pkg_http)[0]
    tar_pkg_cmd = f"cd {PKG_SPACE}; tar xvf {tar_pkg}"
    install_pkg_cmd = f"cd {PKG_SPACE}/full-stack/; ./clean.sh -p all ; ./install.sh -p all ;"
    source_cmd = "/bin/bash -c 'source /etc/profile.d/biren.sh'"
    self.cr.exe_on_terminal(get_pkg_cmd)
    self.cr.exe_on_terminal(tar_pkg_cmd)
    self.cr.exe_on_terminal(install_pkg_cmd)
    self.cr.exe_on_terminal(source_cmd)
```

2.2.2 安装 full-stack 模块流程图

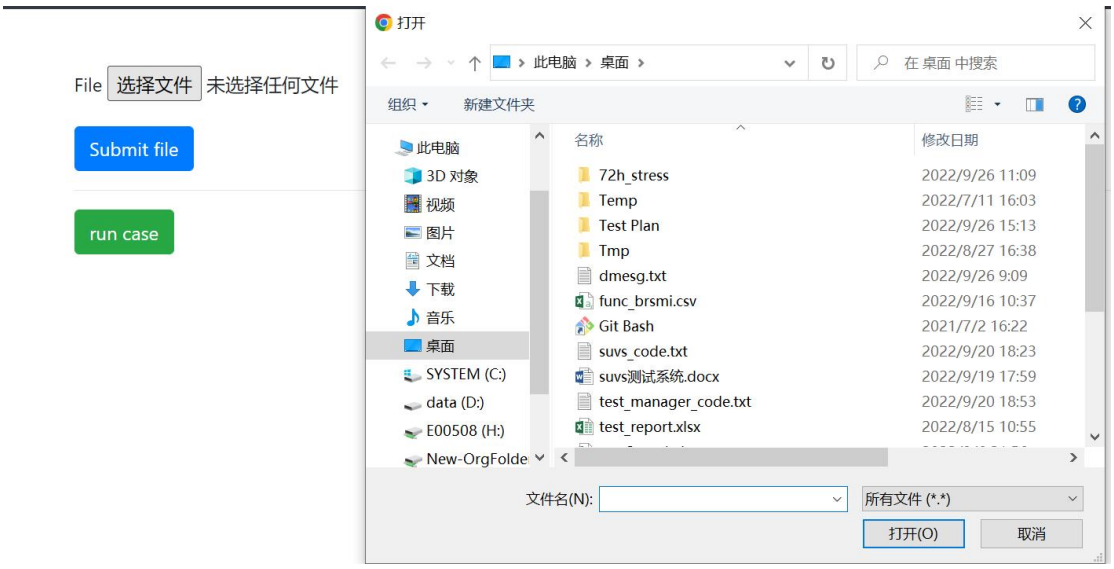


2.3. 上传测试文件

平时我们将测试文件管理在本地，使用 excel 的形式进行存储。方便我们进行管理和存档。进行测试时从 web 端 caserunner 页面上传测试文件，若上传成功，将在当前页面返回 load success。

2.3.1 上传测试文件模块设计

2.3.1.1 上传测试文件前端设计



2.3.1.2 上传文件内容设计

上传的 excel 文件中的每个 sheet 由三个部分组成，分别是：case_id 列，cmd 列，expected 列。三个属性的类型都是 string，可根据需要自由填写。
cmd 填写需要执行的命令行。expected 填写该命令行需要匹配的返回值。其中，expect 可填写为正则字符串。

	A	B	C
1	case_id	cmd	expected
2	1	brsmi | grep SMI	0.6.0
3	2	brsmi | grep Driver	1.0.4
4	3	brsmi | grep SUPA	SUPA Version: 1.0
5	4	brsmi | grep 0000	BR\d{3}-\d{3}W PCIe
6	5	brsmi | grep 0000	\d{8}:\d{2}:\d{2}.\d
7	6	brsmi gpu query | grep Driver	1.0.4
8	7	brsmi gpu query | grep UMD	1.0
9	8	brsmi gpu query | grep SUPA	1.0
10	9	brsmi gpu --query-gpu=name --format csv | grep -A 1 name	BR\d{3}-\d{3}W PCIe
11	10	brsmi gpu --query-gpu=driver_version --format csv | grep -A 1	1.0.4
12	11	brsmi gpu --query-gpu=count --format csv | grep -A 1 count	\d{1,2}

2.3.1.3 上传测试文件 run case 模块设计

- 从前端接收到 case 文件后将存储在 server 端，并同步到远端 remote ip 的 host 上。
- 在远端调用 case runner，解析 excel 文件存储为 dataframe 格式，并批量执行命令行 case，得到匹配结果。

```
def matcher(self,expected,cmd_res) -> str:
    return "passed" if self.rp.reg_match(expected,cmd_res) != [] else "failed"

def run_one_sheet_case(self,sheet_name) -> None:
    df = self.read_the_excel(sheet_name)
    runner = self.cr.cmd_exe_err_str
    df["cmd_res"] = df["cmd"].map(runner)
    df["compare_res"] = df[['expected','cmd_res']].apply(lambda x:self.matcher(x.expected,x.cmd_res), axis=1)

    print(df)
    self.write_to_excel(sheet_name,df)

    series = DataFrame(df["compare_res"]).value_counts()
    ChatGenerator.pie_chart("brsmi",series,"case passing rate")


    # html_text = df.to_html()
    return df
```

- 得到的结果将输出为一个新的 dataframe report，并返回到前端，同时保存为 report excel，后续将提供下载功能。

```
def write_to_excel(self,sheet_name,df) -> None:
    if not os.path.exists(self.report):
        df.to_excel(self.report,sheet_name=sheet_name,index=False)
    else:
        if os.path.exists(self.report):
            book = openpyxl.load_workbook(self.report)
        else:
            book = openpyxl.Workbook()
        writer = pd.ExcelWriter(self.report, engine='openpyxl')
        writer.book = book
        df.to_excel(writer,sheet_name=sheet_name,index=False)
        writer.save()
        writer.close()
```

2.3.2 上传测试文件模块流程图



2.4. 执行测试并同时运行监控程序

2.4.1 执行测试程序模块设计

当点击 run case 后，将运行 excel 中定义的命令行，得到命令行输出，并与期待的返回进行匹配，若匹配成功，返回 pass，若匹配失败，返回 failed。
excel 由三列组成，分别是 case_id、cmd（要执行的命令行）、expected（期待的返回）。

2.4.1.1 excel case 处理模块设计

先用 pandas 批量执行命令得到结果列，再使用 matcher 批量去匹配结果和预期值。正确返回 passed，错误返回 failed，生成 compare 结果列。

```
def matcher(self,expected,cmd_res) -> str:
    return "passed" if self.rp.reg_match(expected,cmd_res) != [] else "failed"

def run_one_sheet_case(self,sheet_name) -> None:
    df = self.read_the_excel(sheet_name)
    runner = self.cr.cmd_exe_err_str
    df["cmd_res"] = df["cmd"].map(runner)
    df["compare_res"] = df[['expected','cmd_res']].apply(lambda x:self.matcher(x.expected,x.cmd_res), axis=1)

    print(df)
    self.write_to_excel(sheet_name,df)

    series = DataFrame(df["compare_res"]).value_counts()
    ChatGenerator.pie_chart("brsmi",series,"case passing rate")


    # html_text = df.to_html()
    return df
```

2.4.1.2 生成 report 并保存

```
def read_the_excel(self,sheet_name) -> None:
    return DataFrame(self.execl_reader.parse(
                                    sheet_name=sheet_name,
                                    header=self.header,
                                    index_col = self.index_col,
                                    dtype = self.dtype))

def write_to_excel(self,sheet_name,df) -> None:
    if not os.path.exists(self.report):
        df.to_excel(self.report,sheet_name=sheet_name,index=False)
    else:
        if os.path.exists(self.report):
            book = openpyxl.load_workbook(self.report)
        else:
            book = openpyxl.Workbook()
        writer = pd.ExcelWriter(self.report, engine='openpyxl')
        writer.book = book
        df.to_excel(writer,sheet_name=sheet_name,index=False)
        writer.save()
        writer.close()
```

2.4.2 在执行 case 的同时运行监控程序

当点击 run case 后，运行 case 的同时，将调用 brsmi 工具监控 silicon 上的 power、clock、memory utilization 及 spc utilization 等情况，并写入 log。

2.4.2.1 monitor 模块设计

可以选择使用 brsmi 工具或使用 brdebug 工具进行监控。

```
def mon_use_brsmi(self,log_file) -> None:
    mon_cmd = f"sudo brsmi gpu --query-gpu=timestamp,pci.bus_id,memory.used,utilization.gpu,utilization.mem,temperature.gpu.current.temp,power.usage,clock.spc,clock.me
    self.cr.exe_need_input(mon_cmd)

def mon_use_brsmi_dmon_multi(self,log_file = "pcuemt") -> None:
    for i in range(self.card.card_num):
        mon_cmd = f"sudo brsmi gpu dmon -i {i} -s pcuemt -o DT -f {BRSMI_LOG}/card_{i}_{log_file}"
        self.cr.exe_need_input(mon_cmd)
    return "mon passed"

def mon_use_br_debug(self,log_name) -> str:
    for i in range(self.card.card_num):
        mon_cmd = f"sudo {self.gb.debuger_path} -i {i} --stat -f {DEBUGGER_LOG}/card_{i}_{log_name} 50"
        self.cr.exe_need_input(mon_cmd)
    return f"{DEBUGGER_LOG}/card_{i}_{log_name}"

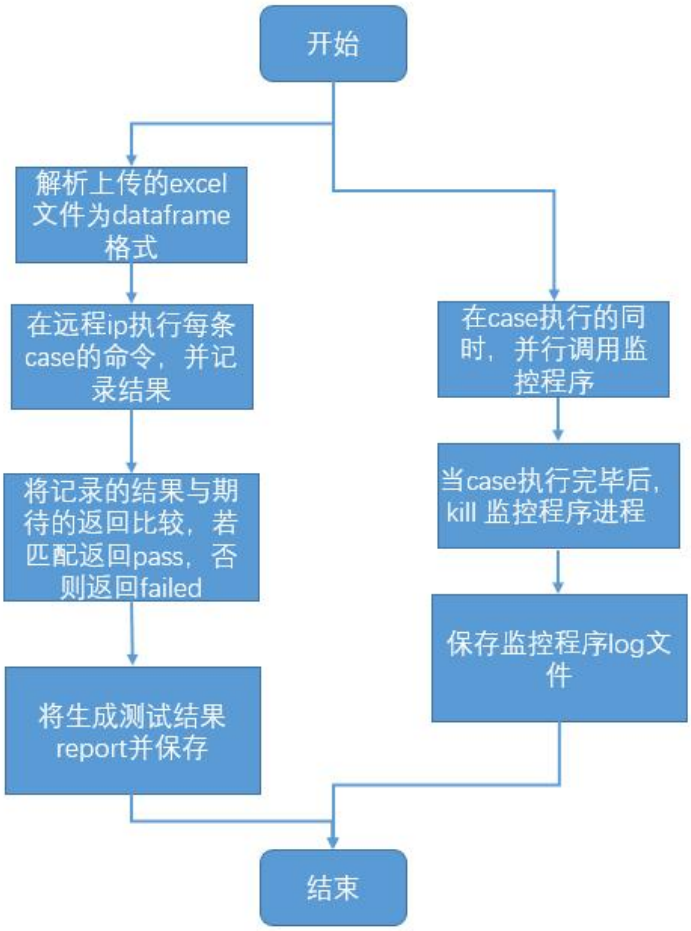
def kill_mon(self,process_name) -> None:
    kill_pid = f"ps -ef|grep {process_name}|grep -v grep|awk '{{print $2}}'|xargs sudo kill -9"
    self.cr.exe_on_terminal(kill_pid)
```

2.4.2.2 case 运行过程中的调用

使用多线程并发地进行调用，在 case 结束时 kill monitor。

```
def run_case_and_mon(self):
    t1 = threading.Thread(target = self.mon_thread_start)
    t1.start()
    passed = self.run_all_sheet_case()
    print(passed)
    self.mon_thread_stop("brsmi")
    self.mon.mon_dmesg_last()
```

2.4.3 测试执行并监控模块流程图



2.5. 展示测试结果

将存在一个中间页面用于展示全部 case 的结果，同时将生成测试结果饼图及监控状态图。

2.5.1 测试结果展示模块设计

2.5.1.1 展示 case 匹配结果

case 的匹配结果由一个中间页面和一个结果饼图展示。case 中间页面可以使用 dataframe 直接展示，也可以用 pandas 提供的 to_html 方法直接展示为 html 表格的形式。

2.5.1.1.1 展示结果表格



/home/jenkins/zxy/app/uploadfile/testCase_1.xlsx
run brsmi cases now

case_id	cmd	expected	cmd_res	compare_res
1	brsmi | grep SMI	0.6.0	| BR-SMI 0.6.3 Driver Version: 1.0.7 ...	failed
2	brsmi | grep Driver	1.0.4	Get process pid failed.\n| BR-SMI 0.6.3 ...	failed
3	brsmi | grep SUPA	SUPA Version: 1.0	| BR-SMI 0.6.3 Driver Version: 1.0.7 ...	passed
4	brsmi | grep 0000	BR\d{3}-\d{3}W PCIe	| 0 BR104-300W PCIe | 00000000:01...	passed
5	brsmi | grep 0000	\d{8}:\d{2}:\d{2}.\d	| 0 BR104-300W PCIe | 00000000:01...	passed
6	brsmi gpu query | grep Driver	1.0.4	Driver Version : 1....	failed
7	brsmi gpu query | grep UMD	1.0	UMD Version : 1....	passed
8	brsmi gpu query | grep SUPA	1.0	SUPA Version : 1....	passed
9	brsmi gpu --query-gpu=name --format csv | gre...	BR\d{3}-\d{3}W PCIe	name\nBR104-300W PCIe \n	passed
10	brsmi gpu --query-gpu=driver_version --format...	1.0.4	driver_version\n1.0.7\n	failed
11	brsmi gpu --query-gpu=count --format csv | gr...	\d{1,2}	count\n1\n	passed
12	brsmi gpu --query-gpu=serial --format csv | g...	\d{2}K\d{7}V\d{5}	serial\n31K069622*V*****\n	failed
13	brsmi gpu --query-gpu=pci.bus_id --format csv...	\d{8}:\d{2}:\d{2}.\d	pci.bus_id\n00000000:01:00.0\n	passed
14	brsmi gpu --query-gpu=pci.bus.domain --format...	0x\d	pci.bus.domain\n0x0\n	passed
15	brsmi gpu --query-gpu=pci.bus --format csv | ...	0x\d	pci.bus\n0x1\n	passed
16	brsmi gpu --query-gpu=pci.bus.device --format...	0x\d	pci.bus.device\n0x0\n	passed
17	brsmi gpu --query-gpu=pci.bus.device_id --for...	1ee0:000f	pci.bus.device_id\n1ee0:000f\n	passed
18	brsmi gpu --query-gpu=pci.sub_system_id --for...	3c61:b148	pci.sub_system_id\n3c61:b148\n	passed

2.5.1.1.2 展示结果饼图

使用 matplotlib 画图，调用以下函数生成饼图：

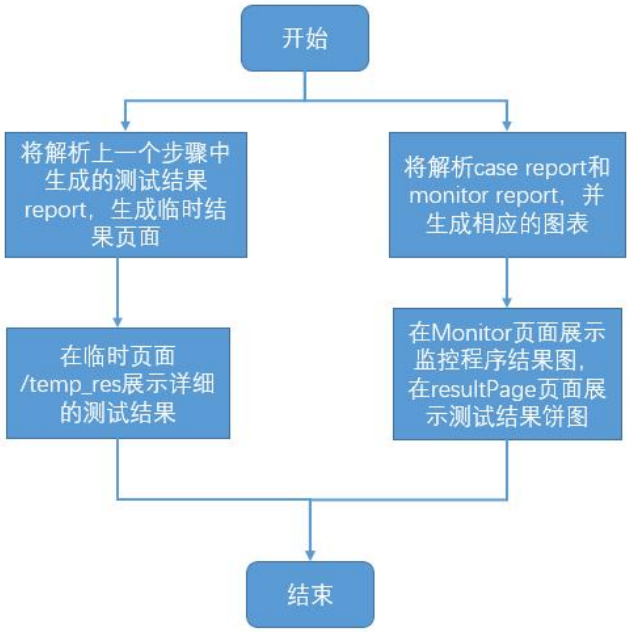
```
@staticmethod
def pie_chart(sheet_name, df, title, suffix=".jpeg"):
    labels = ["passed", "failed"]
    # x = df["compare_res"]
    plt.pie(df, labels=labels ,autopct='%1.1f%%')
    # plt.axis("equal")
    plt.title(title)
    plt.savefig(f"{IMAGES_PATH}/pie_{sheet_name}{suffix}")
    print("save %s pie chat" % sheet_name)
```

2.5.1.2 展示 monitor 结果

先解析 monitor 生成的各卡 csv 结果，保存在 dataframe 中，然后调用 matplotlib 画成折线图，若有多个卡，将展示在同一个坐标系内。

```
def series_line(self,chat_name, se, title, ylable, ylimit=[-5,100], suffix=".jpeg"):
    plt.ylim(ylimit[0], ylimit[1])
    plt.ylabel(ylable,fontsize=12)
    plt.tick_params(axis='both',which='major',labelsize=10)
    y_major_locator= plt.MultipleLocator(10)
    ax=plt.gca()
    ax.yaxis.set_major_locator(y_major_locator)
    plt.tight_layout()
    plt.title(title)
    plt.plot(se)
    plt.savefig(f"{IMAGES_PATH}/series_line_{chat_name}{suffix}")
```

2.5.2 测试执行并监控模块流程图



2.6. 返回测试过程中是否有系统 error

在测试结束后，调用 dmesg 写入 log，并解析 log 中是否包含 aer 等关键 error。若有则展示在 ErrorPage 页面，无则返回“haven't dmesg error”。

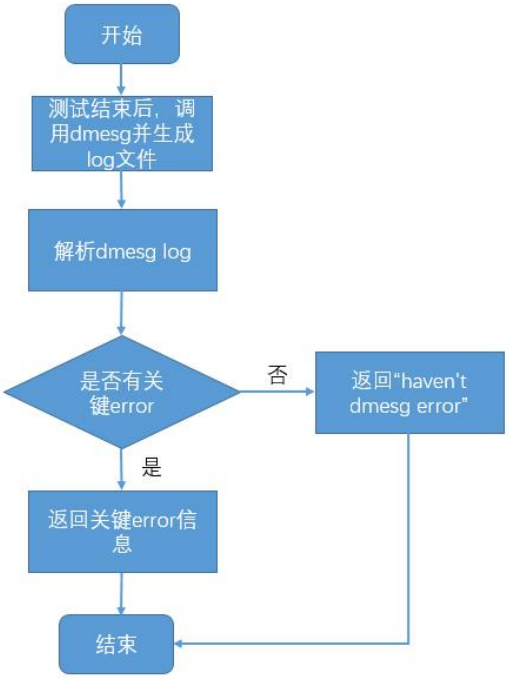
2.6.1 Error 展示模块设计

2.6.1.1 展示是否有系统 error 的模块设计

```
def mon_dmesg_last(self) -> None:
    cmd_content = self.cr.cmd_exe_to_str("dmesg")
    clean_file(f"{ERR_LOG}/dmesg.txt")
    clean_file(f"{ERR_LOG}/dmesg_err.txt")

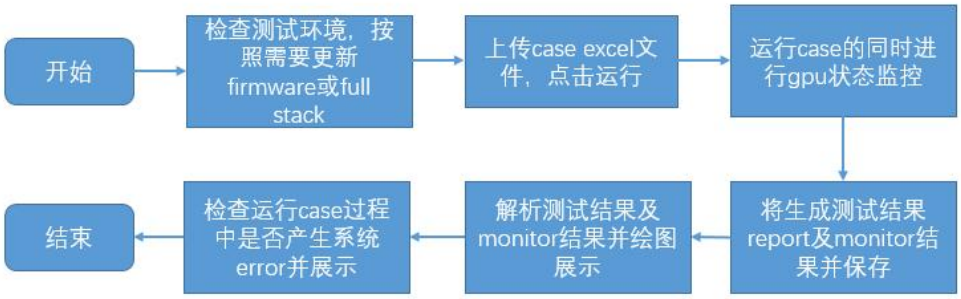
    with open(f"{ERR_LOG}/dmesg.txt","w") as f:
            f.writelines(cmd_content)
    cmd_content1 = cmd_content.split("\n")
    with open(f"{ERR_LOG}/dmesg_err.txt","a+") as ef:
        for line in cmd_content1:
            if "[AER]" in line or "[ERR]" in line:
                line = line + '\n'
                ef.writelines(line)
    lines = str()
    with open(f"{ERR_LOG}/dmesg_err.txt","r") as rf:
        lines = rf.readlines()
    return lines
```

2.6.2 测试执行并监控模块流程图

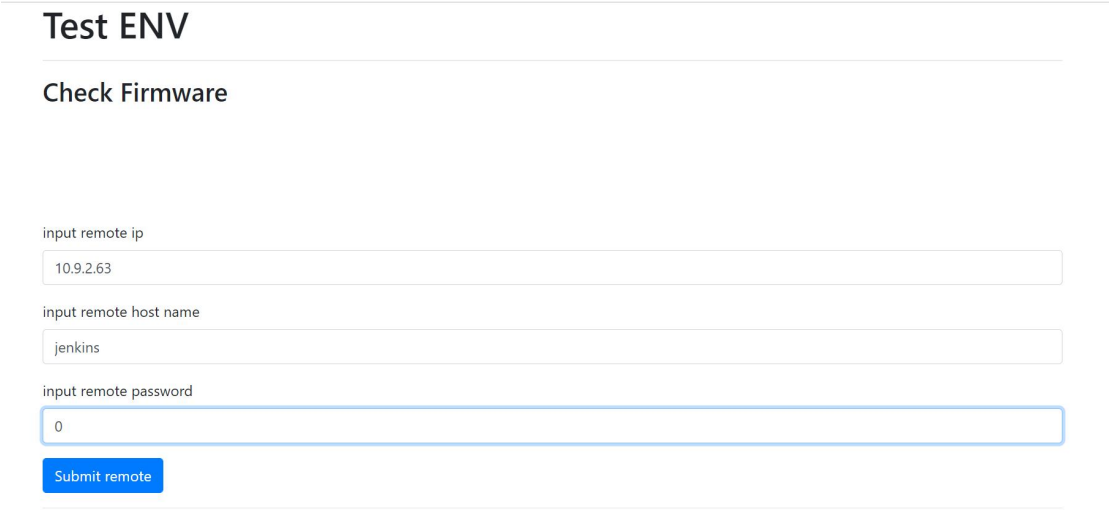

开始
测试结束后，调用dmesg并生成log文件
解析dmesg log
是否有关键error
否
返回“haven't dmesg error”
是
返回关键error信息
结束

3. 简单用例

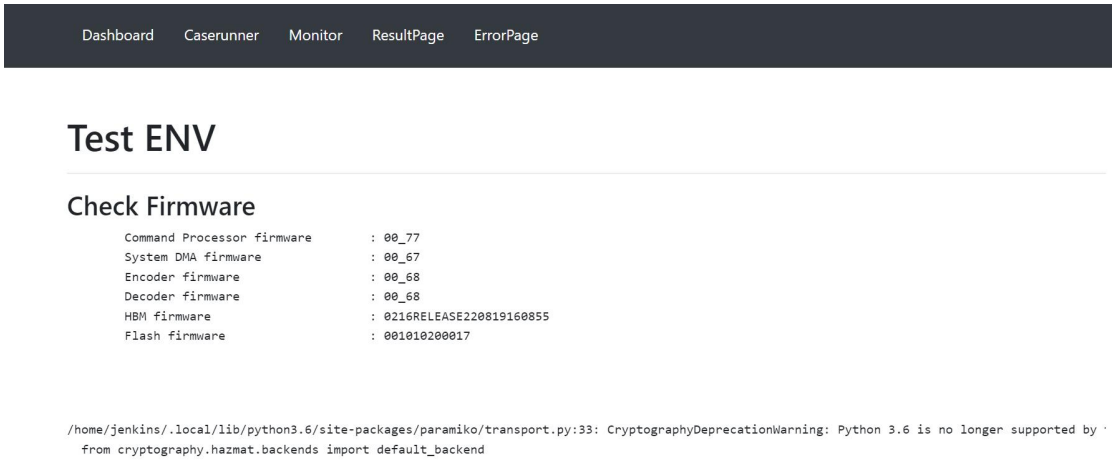
下面展示该平台的使用流程。



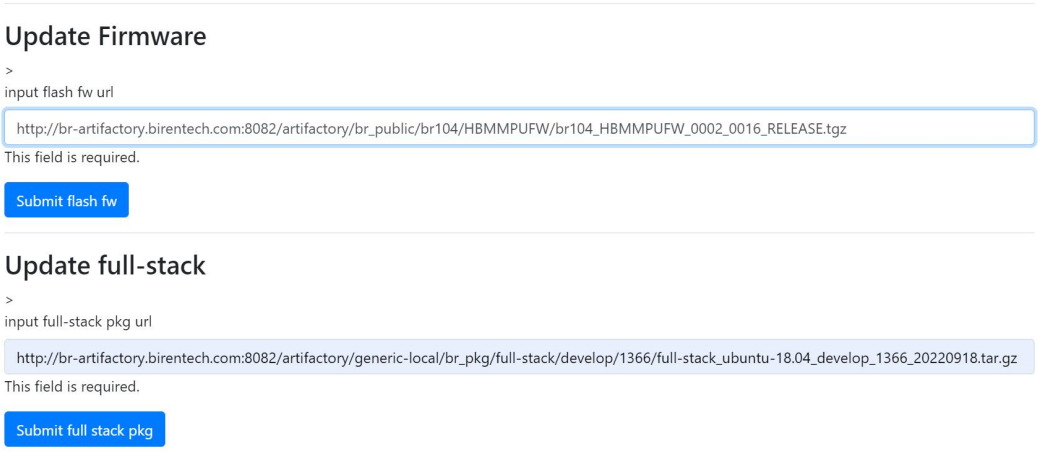
检查测试环境：
输入 remote ip，host name 及 password



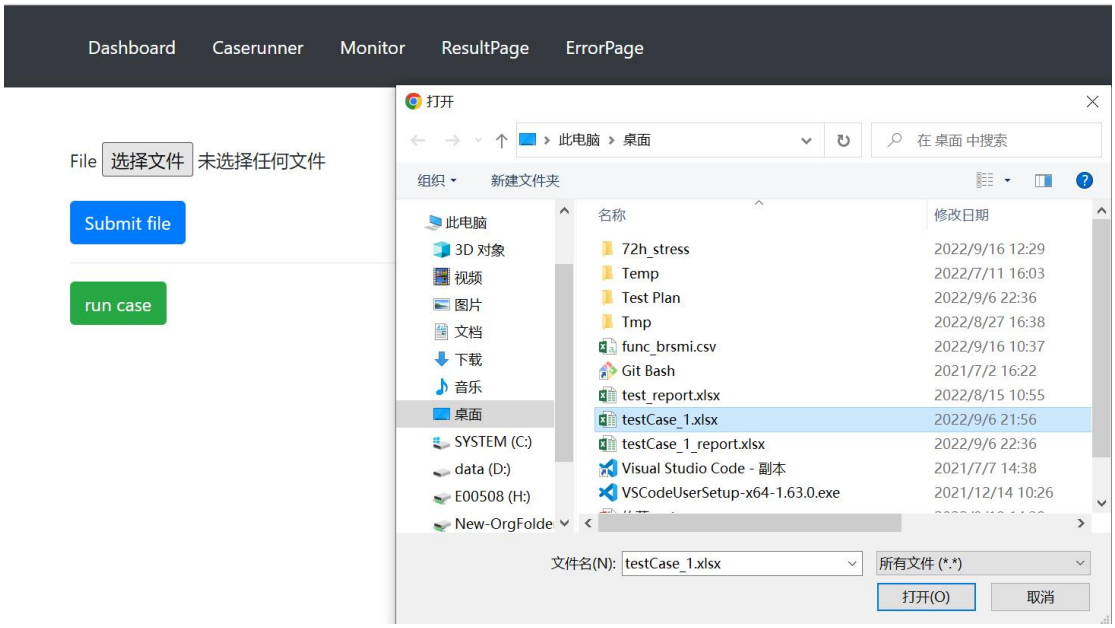
结果展示：将展示查询命令执行结果且捕捉运行中返回的错误



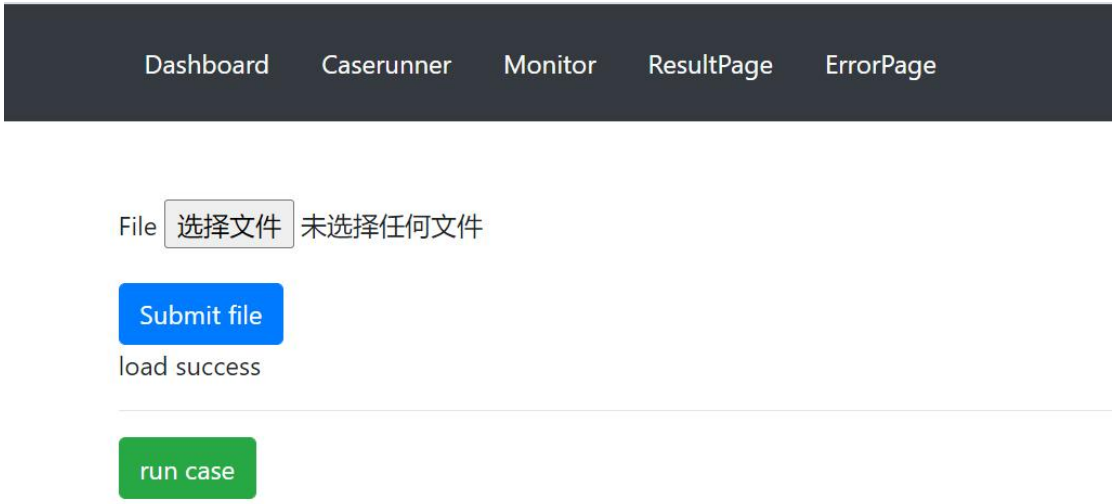
更新 firmware 和 full stack 的部分：



上传 case excel 文件：



上传成功点击 run case



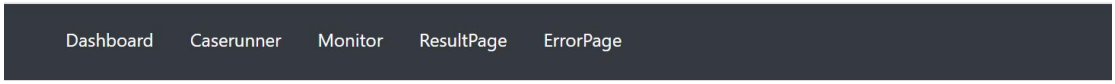
产生临时结果页面：

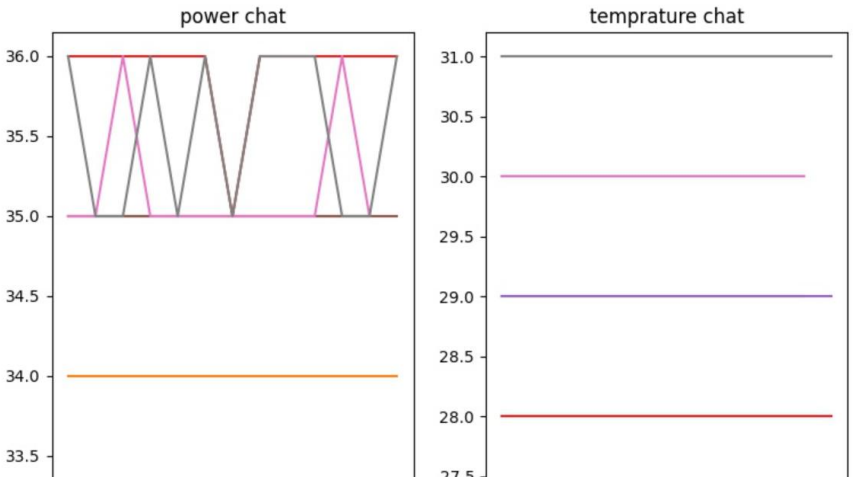


/home/jenkins/zxy/app/uploadfile/testCase_1.xlsx
run brsmi cases now

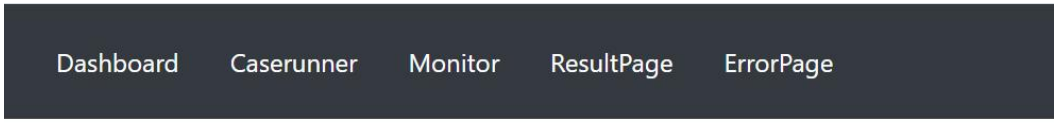
case_id	cmd	expected	cmd_res	compare_res
1	brsmi | grep SMI	0.6.0	| BR-SMI 0.6.3 Driver Version: 1.0.7 ...	failed
2	brsmi | grep Driver	1.0.4	Get process pid failed.\n| BR-SMI 0.6.3 ...	failed
3	brsmi | grep SUPA	SUPA Version: 1.0	| BR-SMI 0.6.3 Driver Version: 1.0.7 ...	passed
4	brsmi | grep 0000	BR\d{3}-\d{3}W PCIe	| 0 BR104-300W PCIe | 00000000:01...	passed
5	brsmi | grep 0000	\d{8}:\d{2}:\d{2}.\d	| 0 BR104-300W PCIe | 00000000:01...	passed
6	brsmi gpu query | grep Driver	1.0.4	Driver Version : 1....	failed
7	brsmi gpu query | grep UMD	1.0	UMD Version : 1....	passed
8	brsmi gpu query | grep SUPA	1.0	SUPA Version : 1....	passed
9	brsmi gpu --query-gpu=name --format csv | gre...	BR\d{3}-\d{3}W PCIe	name\nBR104-300W PCIe \n	passed
10	brsmi gpu --query-gpu=driver_version --format...	1.0.4	driver_version\n1.0.7\n	failed
11	brsmi gpu --query-gpu=count --format csv | gr...	\d{1,2}	count\n1\n	passed
12	brsmi gpu --query-gpu=serial --format csv | g...	\d{2}K\d{7}V\d{5}	serial\n31K069622*V*****\n	failed
13	brsmi gpu --query-gpu=pci.bus_id --format csv...	\d{8}:\d{2}:\d{2}.\d	pci.bus_id\n00000000:01:00.0\n	passed
14	brsmi gpu --query-gpu=pci.bus.domain --format...	0x\d	pci.bus.domain\n0x0\n	passed
15	brsmi gpu --query-gpu=pci.bus --format csv | ...	0x\d	pci.bus\n0x1\n	passed
16	brsmi gpu --query-gpu=pci.bus.device --format...	0x\d	pci.bus.device\n0x0\n	passed
17	brsmi gpu --query-gpu=pci.bus.device_id --for...	1ee0:000f	pci.bus.device_id\n1ee0:000f\n	passed
18	brsmi gpu --query-gpu=pci.sub_system_id --for...	3c61:b148	pci.sub_system_id\n3c61:b148\n	passed

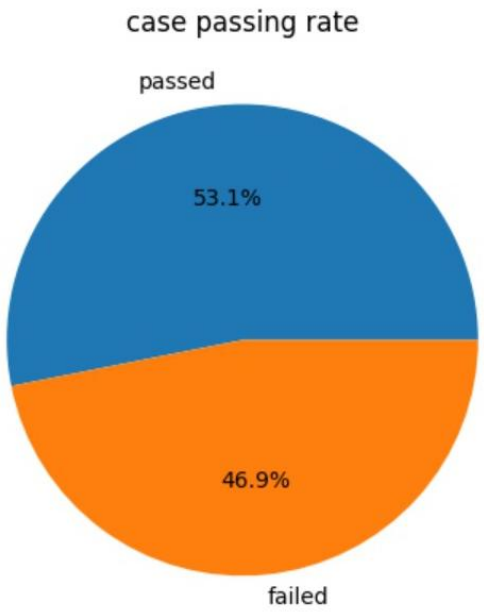
解析 monitor 结果并绘制图片：



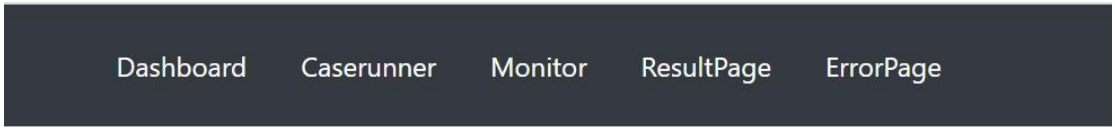


解析 case 结果并绘制结果图片：





查看是否有 error：



haven't dmesg error